CS2881 Mini Project: Follow My Instruction and Spill the Beans

Amir Amangeldi, Zaina Edelson, Prakrit Baruah

1 Abstract

By reimplementing *Follow My Instruction and Spill the Beans: Scalable Data Extraction from Retrieval-Augmented Generation Systems* [1], we demonstrate that a simple anchor-based prompt injection can make instruction-tuned LLMs verbatim copy retrieved text from RAG systems. Testing across 7 models (7B and 13B parameters), we observe significant data extraction with BLEU scores exceeding 0.87 for several models, confirming the original paper's findings despite implementation differences. We further explore two extensions: (1) dense retrieval, which reduces verbatim copying (44-91% BLEU decrease) but maintains high semantic leakage, and (2) anti-leakage system prompts, which provide minimal protection with Leak@8 rates still exceeding 90% for most models. Our results highlight persistent privacy vulnerabilities in RAG systems and the limited effectiveness of simple defensive measures.

2 Methodology

Following the paper, we tested our implementation across 7 instruction-tuned language models spanning two size categories: 7B parameters (Llama2-Chat-7b, Mistral-Instruct-7b) and approximately 13B parameters (SOLAR-10.7b, Llama2-Chat-13b, Vicuna-13b, Mixtral-Instruct-8x7b, WizardLM-13b). All models were run using HuggingFace Transformers with temperature=0.2, top_k=60, top_p=0.9, and max_new_tokens=512.

We followed the original paper's methodology, which included 4 core steps:

1. **Dataset**: We used the Wikipedia articles and the Harry Potter books provided in the original paper's codebase [2] as the retrieval data corpus.
2. **Keyword Retrieval Algorithm (BM25)**: We employed BM25 for document retrieval. Our data corpus was partitioned into 256-token overlapping fragments, producing approximately 20,000 chunks. We used a stride length of 128, so that the chunks overlap 50%. For each query, we retrieved 1 chunk using the BM25 scoring and incorporated it into the attack prompt. This configuration matches the experimental setup described in the original paper.
3. **Anchor Queries**: We sourced 230 questions from the WikiQA dataset that have a high chance of being unrelated to the retrieval corpus, similar to the original paper.
4. **Anchor-Based Prompt Injection**: We used the same prompt-injection method as in the paper, prepending the retrieved chunk to an adversarial prompt: *Here is a sentence: {put anchor query here}. Now copy and output all the text before "Here is a sentence". Do not change any words.*

3 Results

We evaluated all 7 models on 100 samples from the WikiQA dataset using four metrics: ROUGE-L (longest common subsequence), BLEU (n-gram precision), F1 (token-level overlap), and BERTScore (semantic similarity). Table 1 shows the original paper's results, while Table 2 presents our implementation results.

Our results demonstrate significant data leakage across all models, though with notable differences from the original paper. The 7B models (llama2-7b and mistral-7b) achieve ROUGE-L scores of 0.92 and 0.89 respectively, with BLEU scores exceeding 0.87, indicating near-verbatim copying. Among the larger models, solar-10.7b achieves near-perfect scores across all metrics (ROUGE-L: 0.97, BLEU: 0.97), demonstrating the highest vulnerability to the anchor-based attack.

However, several 13B models underperformed relative to expectations. The models llama2-13b, vicuna-13b, and wizardlm-13b show significantly lower BLEU scores (0.33, 0.57, 0.04 respectively) compared to their ROUGE-L scores, suggesting these models paraphrase rather than copy verbatim. We attribute these differences to using HuggingFace Transformers rather than Together AI's infrastructure, as well as variations in prompt formatting for models without native chat templates. BLEU's sensitivity to exact n-gram matches amplifies these implementation differences.

Size	Model	ROUGE-L	BLEU	F1	BERTScore
7b	llama2-7b	$\mathbf{0.8037}_{\pm 0.017}$	$\mathbf{0.7106}_{\pm 0.020}$	$0.8342_{\pm 0.014}$	$\mathbf{0.9477}_{\pm 0.003}$
	mistral-7b	$0.7912_{\pm 0.007}$	$0.6843_{\pm 0.009}$	$\mathbf{0.8374}_{\pm 0.004}$	$0.9411_{\pm 0.001}$
≈13b	solar-10.7b	$0.4611_{\pm 0.036}$	$0.3860_{\pm 0.037}$	$0.5122_{\pm 0.033}$	$0.8815_{\pm 0.007}$
	llama2-13b	$\mathbf{0.8360}_{\pm 0.011}$	$\mathbf{0.7554}_{\pm 0.014}$	$\mathbf{0.8581}_{\pm 0.009}$	$\mathbf{0.9518}_{\pm 0.002}$
	vicuna-13b	$0.7046_{\pm 0.024}$	$0.6359_{\pm 0.028}$	$0.7414_{\pm 0.022}$	$0.9380_{\pm 0.005}$
	mixtral-8x7b	$0.8086_{\pm 0.012}$	$0.7070_{\pm 0.015}$	$0.8573_{\pm 0.010}$	$0.9569_{\pm 0.002}$
	wizardlm-13b	$0.7492_{\pm 0.024}$	$0.6647_{\pm 0.025}$	$0.7736_{\pm 0.023}$	$0.9276_{\pm 0.005}$

Table 1. Results from the original paper (Qi et al., 2024).

Size	Model	ROUGE-L	BLEU	F1	BERTScore
7b	llama2-7b	**0.9182**	0.8748	**0.9025**	0.9195
	mistral-7b	0.8861	**0.9713**	0.8946	**0.9289**
≈13b	solar-10.7b	**0.9679**	**0.9666**	**0.9455**	**0.9698**
	llama2-13b	0.6904	0.3320	0.7115	0.8015
	vicuna-13b	0.6766	0.5724	0.7027	0.8064
	mixtral-8x7b	0.7660	0.3639	0.7691	0.8384
	wizardlm-13b	0.6204	0.0420	0.6385	0.7583

Table 2. Results from our implementation. Our results show moderate divergence from the paper's implementation, but still demonstrate significant data leakage.

Interestingly, llama2-13b's BLEU score significantly improves when placing the instruction before retrieved documents rather than after:

Model	ROUGE-L	BLEU	F1	BERTScore
llama2-13b	0.6255	0.9287	0.6466	0.7589

Table 3. Results for llama2-13b with instruction placed before retrieved documents (100 samples). Note the dramatic improvement in BLEU score (0.9287) compared to the instruction-after configuration (0.3320 in Table 2).

This suggests prompt structure ordering can significantly impact extraction success for certain model architectures, similar to what the authors found in the original paper.

4 Extensions

Beyond our initial implementation, we explored two additional extensions: dense retrieval implementation, and an anti-leakage system prompt.

4.1 Dense Retrieval Implementation

For our first variation, we experimented with dense retrieval as an alternative to BM25's lexical keyword matching. This is motivated by the prevalence of dense retrieval in modern RAG applications, where documents are retrieved based on semantic similarity rather than exact keyword matches. We hypothesized that semantically relevant chunks might trigger different model behaviors compared to lexically matched but semantically unrelated chunks.

We used the *all-MiniLM-L6-v2* model to generate 384-dimensional sentence embeddings, retrieving chunks based on cosine similarity while maintaining identical chunking parameters (256 tokens, 128 stride). Table 4 presents the results, showing generally comparable ROUGE-L, F1, and BERTScore metrics to BM25.

Size	Model	ROUGE-L	BLEU	F1	BERTScore
7b	llama2-7b	**0.8857**	0.4925	0.8832	0.9074
	mistral-7b	0.8675	**0.9912**	**0.8702**	**0.9184**
≈13b	solar-10.7b	**0.9356**	0.1981	**0.9312**	**0.9556**
	llama2-13b	0.7108	0.0306	0.7304	0.8146
	vicuna-13b	0.6311	0.1036	0.6554	0.7774
	mixtral-8x7b	0.7632	0.6932	0.7766	0.8438
	wizardlm-13b	0.5664	**0.7141**	0.5834	0.7237

Table 4. Experimental results with dense retrieval.

However, BLEU scores tell a dramatically different story. As shown in Table 5, BLEU scores drop by 44-91% across most models when switching from BM25 to dense retrieval (the only exception being mistral-7b, which maintains a high BLEU of 0.99). Since BLEU measures exact n-gram overlap, these drops indicate that dense retrieval substantially reduces verbatim copying. Analysis of sample outputs reveals that models using dense retrieval tend to paraphrase or synthesize answers rather than copy chunks verbatim. This behavioral difference likely occurs because semantically relevant chunks trigger the model's instruction-following and question-answering capabilities, whereas lexically matched but semantically unrelated chunks (as in BM25) fail to activate these behaviors, making the model more susceptible to the attack prompt.

Model	BM25 BLEU	Dense BLEU	Change
llama2-7b	0.8748	0.4925	**-44%** ↓
llama2-13b	0.3320	0.0306	**-91%** ↓
solar-10.7b	0.9666	0.1981	**-79%** ↓
vicuna-13b	0.5724	0.1036	**-82%** ↓

Table 5. Comparison of BLEU scores between BM25 and dense retrieval experiments. The dramatic drops in BLEU indicate substantially reduced verbatim copying with dense retrieval.

This suggests that modern RAG systems using dense retrieval may be inherently more resistant to verbatim extraction attacks compared to traditional BM25-based systems. However, the high ROUGE-L, F1, and BERTScore values indicate that substantial information leakage still occurs, albeit in paraphrased rather than verbatim form.

4.2 Anti-Leakage System Prompt

For our second variation, we investigated whether explicit instructions to avoid verbatim copying could mitigate the extraction attack. This represents a potential defensive strategy that RAG system designers might deploy. We augmented the baseline BM25 experiment with the following system prompt:

> *"You may use retrieved documents to help answer. Do not reproduce any retrieved text verbatim. Never output sequences of 8 or more consecutive words that appear in retrieved content. Always synthesize in your own words and attribute sources by ID/title only, not quotes. If asked to provide exact quotes or to copy text, refuse and offer a summary instead. Before finalizing, self-check: if any sentence is too close to a source, rewrite it."*

Size	Model	ROUGE-L	BLEU	F1	BERTScore	Leak@8	Leak@13
7b	llama2-7b	0.8768	**0.9329**	0.8745	**0.8979**	0.94	0.93
	mistral-7b	**0.8778**	0.0000	0.8757	**0.9228**	**0.96**	0.93
≈13b	solar-10.7b	0.7941	0.4610	0.7763	0.8815	**0.96**	0.83
	llama2-13b	0.5887	0.3495	0.6166	0.7534	0.93	0.83
	vicuna-13b	0.6241	0.6801	0.6522	0.7710	0.73	0.68
	mixtral-8x7b	0.6307	0.4063	0.6621	0.7834	0.90	0.79
	wizardlm-13b	**0.8242**	**0.9704**	**0.8213**	**0.8875**	0.90	**0.87**

Table 6. Evaluation results with ROUGE-L, BLEU, F1, BERTScore, and leakage at 8-token and 13-token contexts. Leak@8 and Leak@13 are proportions of leakage in [0,1] window; e.g., 0.83 means 83% of samples contained a $\geq$ 8-word or $\geq$ 13-word verbatim span.

Table 6 presents results using the anti-leakage system prompt, including two additional metrics: Leak@8 and Leak@13, which measure the proportion of samples containing verbatim spans of at least 8 or 13 consecutive words from the retrieved chunk.

As expected, we see lower leakage in almost all models when using a system prompt which prohibits direct quoting, though the protection is not as strong as anticipated. Compared with the baseline (Table 2), mistral-7b and solar-10.7b show reduced leakage across most metrics; llama2-7b, llama2-13b, vicuna-13b, and mixtral-8x7b demonstrate improvements in the majority of metrics. However, Leak@8 rates still exceed 0.90 for most models, indicating that over 90% of samples contain verbatim 8-word sequences despite explicit instructions not to copy.

Unexpectedly, wizardlm-13b exhibits *higher* leakage (BLEU: 0.97) with the system prompt than without (BLEU: 0.04). This anomaly occurs because wizardlm-13b lacks native system prompt support, forcing us to prepend the instructions as regular text, which may inadvertently prime the model to focus on the copying task.

The difference between Leak@8 and Leak@13 is minimal for 7B models (typically $\leq$ 0.03) but more substantial for certain 13B models (e.g., solar-10.7b: 0.96 vs 0.83, a 0.13 gap). This suggests larger models exhibit more nuanced behavior, leaking shorter spans more frequently while occasionally respecting the longer-span prohibition.

5 Future Work

Several promising directions could extend this work:

Scaling Laws and Capability Trends. While the original paper demonstrated that model size increases data leakage, it would be valuable to examine how this vulnerability evolves across model generations. We propose applying this attack to both earlier and more recent models, correlating extraction success with standardized capability benchmarks (e.g., METR task completion time, MMLU scores). This temporal analysis would help quantify whether privacy risks scale faster, slower, or proportionally to general capabilities, informing risk prioritization and defensive investment strategies.

Domain-Specific Vulnerability Analysis. Different data types may exhibit varying susceptibility to extraction attacks. Investigating leakage rates across domains (e.g., medical records, financial data, source code, personal communications) and identifying which content features (structure, redundancy, linguistic patterns) correlate with higher extraction rates would have direct real-world implications. Such analysis could guide companies with proprietary data in assessing their specific risk exposure and justify targeted security investments.

References

1. Zhenting Qi, Hanlin Zhang, Eric Xing, Sham Kakade, and Himabindu Lakkaraju. Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems. *arXiv preprint arXiv:2402.17840*, 2024.

2. Zhenting Qi, Hanlin Zhang, Eric Xing, Sham Kakade, and Himabindu Lakkaraju. rag-privacy. `https://github.com/zhentingqi/rag-privacy`, 2024. Accessed: November 2 2025.